DALHOUSIE UNIVERSITY

Performance Estimation (cont.)

Time-ordered data, nesting, and comparing models

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Can you trust the score?

What it is for Holdout, random subsampling, cross-validation and the bootstrap, as previously seen, turn a model into a performance estimate. This class asks whether that estimate deserves to be believed, and names three common ways it does not.

1. **The split saw the future.** Shuffled folds let a time series train on observations that come after the ones it is tested on (Section 2).
2. **The search got lucky.** The best score of a hyper-parameter search is partly the luckiest score, so it is too high (Section 3).
3. **The winner won by luck.** One model scoring higher than another may be noise, and the obvious tests overstate the evidence (Section 4).

Each trap comes with its fix.

Setup code Every listing in this handout runs, top to bottom in a fresh session, after this setup.

R

```r
library(caret)
```

Python

```python
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.datasets import (
    load_breast_cancer,
    load_iris,
)
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis,
)
from sklearn.model_selection import (
    GridSearchCV,
    KFold,
    StratifiedKFold,
    TimeSeriesSplit,
    cross_val_score,
)
from sklearn.neighbors import (
    KNeighborsClassifier,
    KNeighborsRegressor,
)
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

knn5 = make_pipeline(StandardScaler(),
                     KNeighborsClassifier(5))
lda  = LinearDiscriminantAnalysis()
```

2 The split saw the future

What it is for Every resampling method seen so far shuffles or resamples rows at random, and so treats them as independent draws. Time-ordered rows share a clock, and a time series forbids that. This section gives the replacements.

Why random resampling fails In a time series each value depends on the ones before it. If a shuffled test block sits in the middle of the series, its training set contains observations from both before and after it, and the model is graded on a question whose answer it has partly seen. The resulting score

can look excellent and means little, because a deployed model never sees the future. The rule follows directly. Train only on the past, and test only on the future.

Simple train-test windows Divide the data into past data (observations up to a time t) and future data (observations after t). Figure 1 draws the three schemes.

- **Fit once.** One model is fitted on the training data and applied to the whole test period. It is the cheapest scheme, and it never learns from newer data.
- **Growing window.** Every w_{test} test cases, a new model is fitted on all data up to that point. The training window grows.
- **Sliding window.** Every w_{test} test cases, a new model is fitted on the previous w_{train} observations. The training window has fixed length and moves forward, discarding the oldest data.

A growing window assumes the relationship is stable, so more history helps. A sliding window assumes the relationship drifts, so old data mislead.

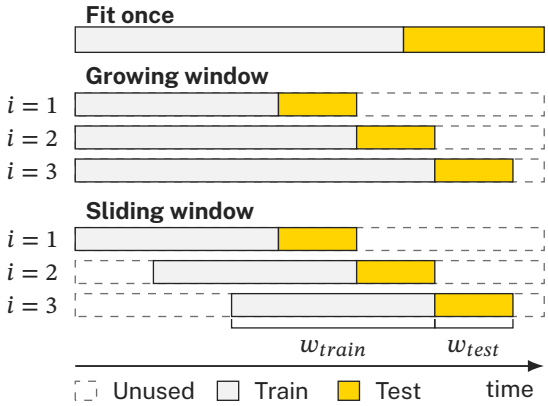


Figure 1. Fit once, growing and sliding train-test windows. Training always precedes testing.

Random subsampling along time One train-test cut gives one noisy score. Random subsampling averages many random splits, and the time-ordered version averages many random cut points, each with its training window before its test window.

1. Select a training window size w_{train} and a testing window size w_{test}.
2. Randomly generate r points $s_1, \ldots, s_r$ within the interval $[w_{train}, \ldots, (n - w_{test})]$.
3. For each point, train on the previous w_{train} observations and test on the next w_{test} observations.
4. Average the r test scores.

The interval leaves room for both windows. A point closer than w_{train} to the start has no room to train, and one closer than w_{test} to the end has no room to test. To tune hyper-parameters, place an internal train-test window inside each training window, which is nesting applied along the time axis. Cerqueira, Torgo, and Mozetič [3] compare these estimators empirically across many real series.

Worked example How much does shuffling flatter a time series? The listings below simulate a random walk of 200 days and predict each day's value from its date alone, with 5-NN regression. Shuffled 10-fold cross-validation reports a mean squared error of 0.78 in R and 0.75 in Python. A sliding window that trains on 100 days and tests on the next 10, moved forward ten times, reports 6.37 and 8.46. The

shuffled score is about ten times too good, because every shuffled test day has training days on both sides of it and kNN only has to copy its neighbours. Walking forward, the model must forecast, which is the job it will have in use. The two languages simulate different walks, so compare the pattern, not the digits.

Shuffled folds against walking forward in R and Python

R

```r
set.seed(3141)                 # a random walk
n <- 200
d <- data.frame(t = 1:n, y = cumsum(rnorm(n)))
mse <- function(tr, te) {      # kNN on the date
  f <- knnreg(y ~ t, d[tr, ], k = 5)
  mean((predict(f, d[te, ]) - d$y[te])^2)
}
shuffled <- createFolds(d$y, k = 10)
forward <- createTimeSlices(1:n,
  initialWindow = 100, horizon = 10,
  fixedWindow = TRUE, skip = 9)   # sliding
mean(sapply(shuffled,
  function(te) mse(-te, te)))     # 0.78
mean(mapply(mse, forward$train,
  forward$test))                  # 6.37
```

Python

```python
rng = np.random.default_rng(3141)  # a walk
t = np.arange(1, 201).reshape(-1, 1)
y_t = np.cumsum(rng.normal(size=200))
knn_t = KNeighborsRegressor(5)      # on the date
def mse(cv):
    return -cross_val_score(knn_t, t, y_t, cv=cv,
        scoring="neg_mean_squared_error").mean()
print(mse(KFold(10, shuffle=True,
                random_state=3141)))   # 0.75
fwd = TimeSeriesSplit(10, test_size=10,
                      max_train_size=100)
print(mse(fwd))                        # 8.46
```

Differences to know

- `TimeSeriesSplit` never shuffles. On an ordinary table that would be a fault, and for a time series it is exactly what you want.
- `max_train_size` turns its growing window into a sliding one.
- caret's `createTimeSlices` calls a sliding window `fixedWindow = TRUE`, because the window's length is fixed. It is not the fit-once scheme above.

3 The search got lucky

What it is for Nesting, as previously seen, runs the hyper-parameter search inside the evaluation, and the best inner score must not be reported. This section shows why, and measures how far that score is off.

The intuition Sixteen students guess every answer on a multiple-choice quiz. Their scores scatter around the guessing rate, and the top score sits well above it, although no student knows more than any other. Reporting the top score as "how well a guesser does" would overstate it.

The rule A grid search is the same experiment. If you search over M candidate settings and report the best inner accuracy, you have reported the maximum of M noisy estimates. The maximum is biased upward *even when every setting is equally good*, because the search picks whichever setting happened to suit these particular folds best. The bias grows with M and with the noise in each estimate [2]. **Nested resampling** fixes it. Choose inside, score outside, and let the outer loop treat the search as part of the model.

Worked example A kNN classifier on standardized `iris` features, with $k \in \{1, 3, 5, \ldots, 31\}$ (sixteen candidates), tuned by 4-fold inner cross-validation and assessed by a 4-fold outer loop, over twenty trials with different seeds. Table 1 gives the optimism and Figure 2 plots every trial.

Table 1. Nested versus non-nested cross-validation optimism for tuned kNN on iris

	mean over 20 trials
Non-nested score (best inner score)	0.9641
Nested score (outer 4-fold loop)	0.9477
Optimism	**0.0164**

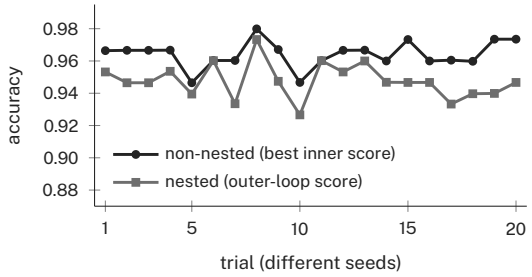


Figure 2. Non-nested (best inner score) and nested accuracy over 20 seeds

The non-nested score is above the nested one on 19 of the 20 trials and equal on trial 11, by as much as 0.0336 on trial 19. The mean optimism is 1.64 accuracy points. A score can also look better by chance alone, with no search at all, and Section 4 turns to that. The listings below run trial 1.

R

```r
set.seed(3141)
inner <- trainControl(method = "cv", number = 4)
outer <- createFolds(iris$Species, k = 4,
                     returnTrain = TRUE)
grid  <- data.frame(k = seq(1, 31, 2)) # 16
nested <- sapply(outer, function(tr) {
  m <- train(Species ~ ., data = iris[tr, ],
             method = "knn", tuneGrid = grid,
             preProcess = c("center", "scale"),
             trControl = inner)   # chooses INSIDE
  p <- predict(m, iris[-tr, ])    # scores OUTSIDE
  mean(p == iris$Species[-tr])
})
mean(nested)                      # 0.9602
```

Python

```python
X, y = load_iris(return_X_y=True)
knn   = make_pipeline(StandardScaler(),
                      KNeighborsClassifier())
grid  = {"kneighborsclassifier__n_neighbors":
         list(range(1, 32, 2))}  # 16 values
inner = StratifiedKFold(4, shuffle=True,
                        random_state=0)
outer = StratifiedKFold(4, shuffle=True,
                        random_state=0)
gs = GridSearchCV(knn, grid, cv=inner)
# the selected inner maximum, NOT an estimate
print(gs.fit(X, y).best_score_)          # 0.9664
# the outer loop scores the whole search
print(cross_val_score(gs, X, y,
                      cv=outer).mean())  # 0.9532
```

Differences to know

- In Python, pass the `GridSearchCV` object as the estimator to `cross_val_score`. The outer loop then refits the whole search on every outer training fold.
- In `caret`, `trainControl` is the inner loop and the outer loop is a loop you write around `train`. Its folds differ from Python's, so its outer estimate is 0.9602.
- `gs.best_score_` is a selected inner maximum, not an outer estimate. Quoting it as the model's performance is a reporting error.

4 The winner won by luck

What it is for Two models are scored on the same data set and one scores higher. This section asks whether the gap is real or could be luck, and shows why the obvious answer overstates the evidence.

The question A 5-nearest-neighbour classifier on standardized features and LDA are scored on `digits` [1], 1797 handwritten digits stored as 8×8 pixel images, so 64 predictors and 10 classes, over one shared stratified 10-fold partition (`StratifiedKFold(n_splits=10, shuffle=True, random_state=3141)`). Because both models see the same folds, each fold is a fair head-to-head. Figure 3 shows every fold. 5-NN averages 0.977 and LDA 0.953, a gap of 2.4 points, and 5-NN wins all ten folds. Is that proof?

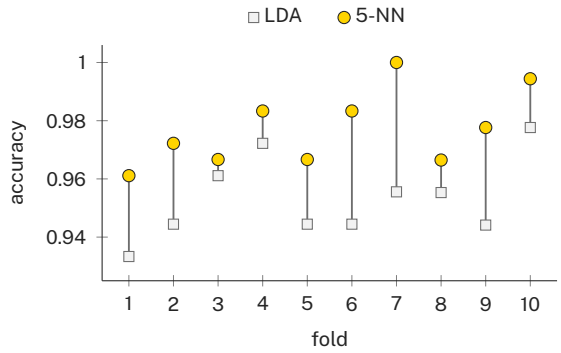


Figure 3. Fold-by-fold accuracy of 5-NN and LDA on digits. Each stem joins the two scores of one fold, and its length is the paired difference d_i.

4.1 The idea of a test

Coin flips Suppose the two models are equally good. This assumption is the **null hypothesis**. Then each fold is a coin flip, and either model wins it half the time. All ten folds going to 5-NN, or all ten to LDA, would then happen with probability

$$p = 2 \times \frac{1}{2^{10}} = \frac{2}{1024} = 0.002.$$

That probability is the **p-value**, the probability, if the null hypothesis were true, of a result at least as extreme as the one observed. A small p-value says the observed result would be surprising if the models were equally good. If it falls below a threshold chosen before looking at the results, usually 0.05, the null hypothesis is rejected and the gap is called **significant**.

Using the sizes of the gaps Counting wins ignores how large each win is. The **paired t-test** uses the size. It measures the mean gap in standard errors, how far luck alone would move it. With d_i the gap on fold i, $\bar{d}$ their mean and s_d their standard deviation over n folds,

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} = \frac{0.02392}{0.01285/\sqrt{10}} = 5.889,$$

If the two models were equally good, this t would follow a t distribution with $n - 1 = 9$ degrees of freedom, where values beyond ± 2.26 occur only 5 percent of the time. 5.889 is far beyond that, and the area past ± 5.889 is $p = 0.000232$, the number R's `t.test` and SciPy's `ttest_rel` return. A mean gap almost six standard errors from zero is hard to explain by luck, and the coin flips agree.

4.2 What a p-value does and does not say

Table 2 sets the two side by side. A p-value measures surprise under an assumption. It is not the probability that one model is better, and it says nothing about how large the gap is, so report the gap beside it.

Table 2. What a p-value does and does not say about a model difference

It says	It does not say
If the two models were truly equivalent, a difference at least this extreme would arise with probability p.	That the null hypothesis is true with probability p.
Under a valid sampling model, small p makes random sampling variation an uncomfortable explanation.	That the difference is *large*. With enough rows, or enough repeated folds, a tiny difference can produce a misleadingly small naive p.
	That the result will replicate on a different data set. A single data set tests one population, not the method.
	Anything inferential if models were not paired on the same splits, or if resampling dependence was ignored.

4.3 Ten folds are not ten coins

Both calculations above assume the ten folds are independent, like ten flips of a coin. They are not, and as previously mentioned $s/\sqrt{K}$ is not an ordinary standard error for this reason. In 10-fold cross-validation any two training sets share 8 of their 9 blocks of rows, as Figure 4 draws. The ten fitted models are near copies, so when one fold favours 5-NN the others tend to as well. Ten correlated folds carry less evidence than ten independent coins, and a test that treats them as independent is overconfident. Pairing the folds removes each fold's own difficulty from the comparison. It does not make the folds independent.

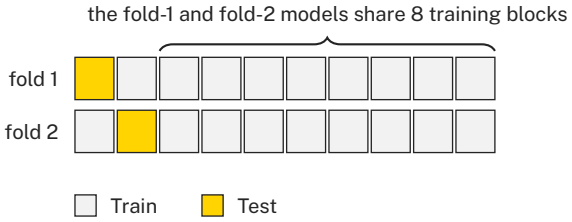


Figure 4. Two folds of one 10-fold cross-validation. Their training sets share 8 of their 9 blocks.

The proof The `digits` gap is too clear to show the trap, so take a closer race. `breast_cancer` [8, 10] holds 569 biopsies with 30 measurements each, and 5-NN beats LDA on it by 1.2 accuracy points. Table 3 and Figure 5 test that gap three ways. On one 10-fold run the paired t-test gives $p = 0.111$, not significant. Repeating the 10-fold CV ten times and testing the 100 differences as if they were independent gives $p = 1.4 \times 10^{-5}$. The mechanism is the denominator. The standard error divides by $\sqrt{n}$, and $\sqrt{n}$ grew from 3.2 to 10, so t rose from 1.77 to 4.58, helped by a slightly smaller mean gap on the repeated runs, 0.0107 against 0.0123. Nothing new was learned, because the 100 folds reuse the same 569 rows, yet the naive p-value fell by four orders of magnitude and crossed every conventional threshold. Repeating cross-validation makes the estimate steadier. It does not add evidence.

Table 3. Naive versus corrected resampled t-test across repeated cross-validation

t-test	n	$\bar{d}$	t	p
10-fold CV, naive	10	0.0123	1.769	0.111
10×10 CV, naive	100	0.0107	4.576	1.4×10^{-5}
10×10 CV, corrected	100	0.0107	1.315	0.192

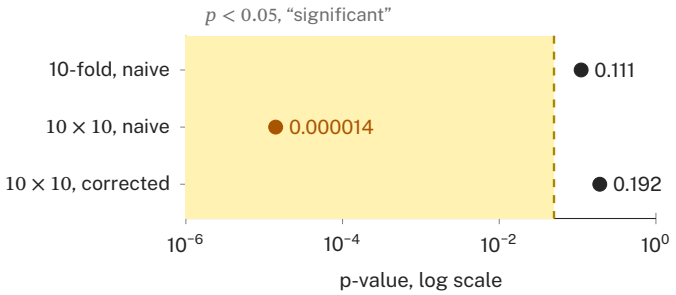


Figure 5. The three p-values of Table 3 on a log scale. The shaded band is where a test at 0.05 rejects.

4.4 Paying for the overlap

The corrected resampled t-test Nadeau and Bengio [5] keep the paired t-test and widen its standard error to pay for the overlap. The usual standard error $s_d\sqrt{1/n}$ becomes

$$s_d \sqrt{\frac{1}{n} + \frac{n_{test}}{n_{train}}},$$

where n_{test}/n_{train} is the number of test rows per training row in one split. In K-fold CV each fold tests one row for every $K-1$ it trains on, so the term is $1/9$ for 10 folds and $1/4$ for 5. The usual term $1/n$ shrinks as the folds are repeated, and the overlap term does not, so repetition no longer buys significance. Since only the standard error changes, the corrected t is the naive one times a factor fixed by the design,

$$t_{corrected} = t_{naive} \times \sqrt{\frac{1/n}{1/n + n_{test}/n_{train}}},$$

which is 0.69 for one 10-fold run and 0.29 for 10×10. The p-value is then read from the same t distribution with $n-1$ degrees of freedom, which `pt` in R and `stats.t.sf` in SciPy do.

Reading the result On `breast_cancer` the correction brings the 10×10 p-value back to 0.192, so the 1.1-point gap of the repeated runs could be luck. On `digits` the corrected test gives $t = 4.053$ and $p = 0.0029$, so luck is an unlikely explanation for the 2.4-point advantage of 5-NN over LDA. The listings below compute the coin-flip test, the paired t-test and the corrected test on the `digits` folds. Both languages type in the same counts of correct predictions per fold, out of 179 or 180 test images, so they test identical numbers.

R

```r
n <- c(rep(180, 7), rep(179, 3))  # test images
knn <- c(173, 175, 174, 177, 174,
         177, 180, 173, 175, 178) / n
lda <- c(168, 170, 173, 175, 170,
         170, 172, 171, 169, 175) / n
d <- knn - lda
binom.test(sum(d > 0), length(d))
#   p-value = 0.001953, coin flips (naive)
t.test(knn, lda, paired = TRUE)
#   t = 5.8891, p-value = 0.0002322 (naive)
```

Python

```python
n = np.array([180] * 7 + [179] * 3)  # test images
knn = pd.Series([173, 175, 174, 177, 174,
                 177, 180, 173, 175, 178]) / n
lda = pd.Series([168, 170, 173, 175, 170,
                 170, 172, 171, 169, 175]) / n
d = knn - lda
print(stats.binomtest(int((d > 0).sum()),
                      len(d)).pvalue)
# 0.001953, ten coin flips (naive)
print(stats.ttest_rel(knn, lda))
# statistic 5.889, pvalue 0.000232 (naive)
```

R

```r
corrected_t <- function(d, ratio) { # test/train
  v <- (1 / length(d) + ratio) * var(d)
  t <- mean(d) / sqrt(v)
  c(t = t, p = 2 * pt(-abs(t), length(d) - 1))
}
corrected_t(d, 1 / 9)     # t 4.053, p 0.00287
```

Python

```python
def corrected_t(d, ratio):  # n_test / n_train
    v = (1 / len(d) + ratio) * d.var(ddof=1)
    t = d.mean() / np.sqrt(v)
    return t, 2 * stats.t.sf(abs(t), len(d) - 1)
print(corrected_t(d, 1 / 9))     # 4.053 0.00287
```

4.5 Same folds for every model

The rule Every model is scored on the same folds. If two models are scored on *different* random partitions, the differences d_i pair nothing and every test in this class is invalid. Each model's own estimate is still fine; only the *comparison* fails. Neither library raises an error. The breast-cancer data in R come in a different version, so the R listing pairs the two models on `iris`, where LDA leads 5-NN by 2.7 points.

R

```r
set.seed(3141)
folds <- createFolds(iris$Species, k = 10,
                     returnTrain = TRUE)
ctrl  <- trainControl("cv", index = folds)
m_knn <- train(Species ~ ., iris, "knn",
               preProcess = c("center", "scale"),
               tuneGrid = data.frame(k = 5),
               trControl = ctrl)
m_lda <- train(Species ~ ., iris, "lda",
               trControl = ctrl)
identical(m_knn$control$index,
          m_lda$control$index)           # TRUE
rs <- resamples(list(kNN = m_knn,
                     LDA = m_lda))
# resamples() matches fold labels only
v <- rs$values
mean(v[["kNN~Accuracy"]] - v[["LDA~Accuracy"]])
# -0.027, LDA ahead on these folds
```

Python

```python
X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(10, shuffle=True,
                     random_state=3141)
# ONE splitter object, reused for every workflow
a = cross_val_score(knn5, X, y, cv=cv)
b = cross_val_score(lda,  X, y, cv=cv)
# WRONG: a fresh partition per call, no warning
# cross_val_score(knn5, X, y,
#     cv=StratifiedKFold(10, shuffle=True))
splits = list(cv.split(X, y))  # safest: freeze
a = cross_val_score(knn5, X, y, cv=splits)
print(a[:3].round(4))   # 0.9825 0.9474 0.9474
print((a - b).mean().round(4))   # 0.0123
```

Differences to know

- In R one `index` object is passed through `trainControl` to every `train` call, which fixes the folds. `resamples()` checks only that the models carry the same number of resamples, matched by fold label. Two fits made with different seeds carry the same labels (`Fold01`, ...) on different partitions and pass without an error, so compare the `index` objects as the listing does.

- In Python the pairing is your responsibility. A shuffled splitter built without a `random_state` reshuffles on every call, so even one splitter object passed to two models gives them unrelated partitions. Set `random_state`, or freeze the splits in a list once, and pass that to every model.

5 Common misconceptions

1. **"I will just use cross-validation on my time series."** Random folds train on the future. Fit once, or use a growing, sliding or random window, always training on the past.
2. **"`best_score_` is the model's performance."** It is the maximum of M noisy estimates, 1.64 accuracy points too high on average in the `iris` example. Only an outer loop that contains the search produces an estimate.
3. **"It won all ten folds, so the win is proven."** The coin-flip p-value of 0.002 assumes independent folds, and the folds share 8 of 9 training blocks. The corrected test is the one to trust.
4. **"Repeating cross-validation makes the result more significant."** It makes the *estimate* more stable and the naive *test* invalid. The p-value moved from 0.111 to 1.4×10^{-5} on identical evidence.
5. **"It wins on this data set, so it is the better method."** 5-NN beats LDA on `digits` even after the correction. That is a statement about `digits`, and another data set can reverse it.

6 Summary

- Time-ordered rows need fit-once, growing, sliding or random windows, in which training always precedes testing.
- The best inner score of a search is biased upward. Report the outer estimate of a nested search.
- Score every model on the same folds. Folds overlap, so naive fold-level tests are overconfident, and repeating cross-validation does not add evidence.
- A p-value measures how surprising a result would be if the models were equal. It is not the probability that one model is better. Report the size of the gap beside it.
- To compare two models, score both on the same folds and use the corrected resampled t-test, which pays for the overlap.

References

[1] E. Alpaydin and C. Kaynak. *Optical Recognition of Handwritten Digits*. UCI Machine Learning Repository. Dataset. 1998.

[2] G. C. Cawley and N. L. C. Talbot. "On over-fitting in model selection and subsequent selection bias in performance evaluation". In: *Journal of Machine Learning Research* 11.70 (2010), pp. 2079–2107.

[3] V. Cerqueira, L. Torgo, and I. Mozetič. "Evaluating time series forecasting models: an empirical study on performance estimation methods". In: *Machine Learning* 109.11 (2020), pp. 1997–2028.

[4] M. Kuhn. "Building predictive models in R using the caret package". In: *Journal of Statistical Software* 28.5 (2008), pp. 1–26.

[5] C. Nadeau and Y. Bengio. “Inference for the generalization error”. In: *Machine Learning* 52.3 (2003), pp. 239–281.

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. “Scikit-learn: machine learning in Python”. In: *Journal of Machine Learning Research* 12.85 (2011), pp. 2825–2830.

[7] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria, 2026.

[8] W. N. Street, W. H. Wolberg, and O. L. Mangasarian. “Nuclear feature extraction for breast tumor diagnosis”. In: *Biomedical Image Processing and Biomedical Visualization*. Vol. 1905. Proceedings of SPIE. SPIE, 1993, pp. 861–870.

[9] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, et al. “SciPy 1.0: fundamental algorithms for scientific computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272.

[10] W. Wolberg, O. Mangasarian, N. Street, and W. Street. *Breast Cancer Wisconsin (Diagnostic)*. UCI Machine Learning Repository. Dataset. 1993.